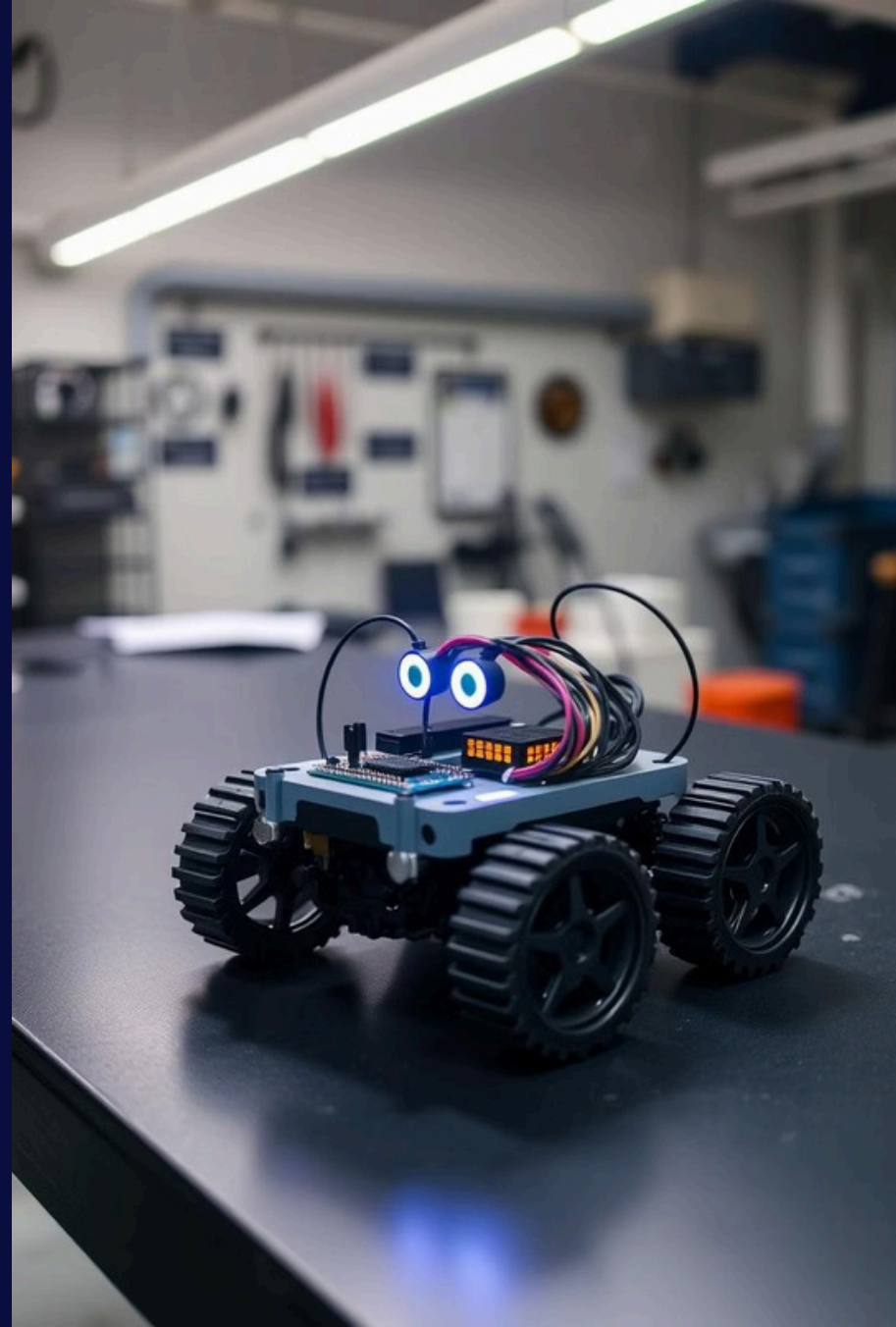


Robot Éviteur d'Obstacles avec Arduino

Découvrez comment créer un robot capable de naviguer seul et d'éviter les obstacles. Ce projet Arduino combine matériel et algorithmie pour une mobilité autonome fiable.



by : MOHAMED SOUBHI
ANGE KOYOU



Composants Matériels Essentiels

Carte Arduino Uno R3

Microcontrôleur principal pour gérer les capteurs et moteurs.

Capteur à ultrasons HC-SR04

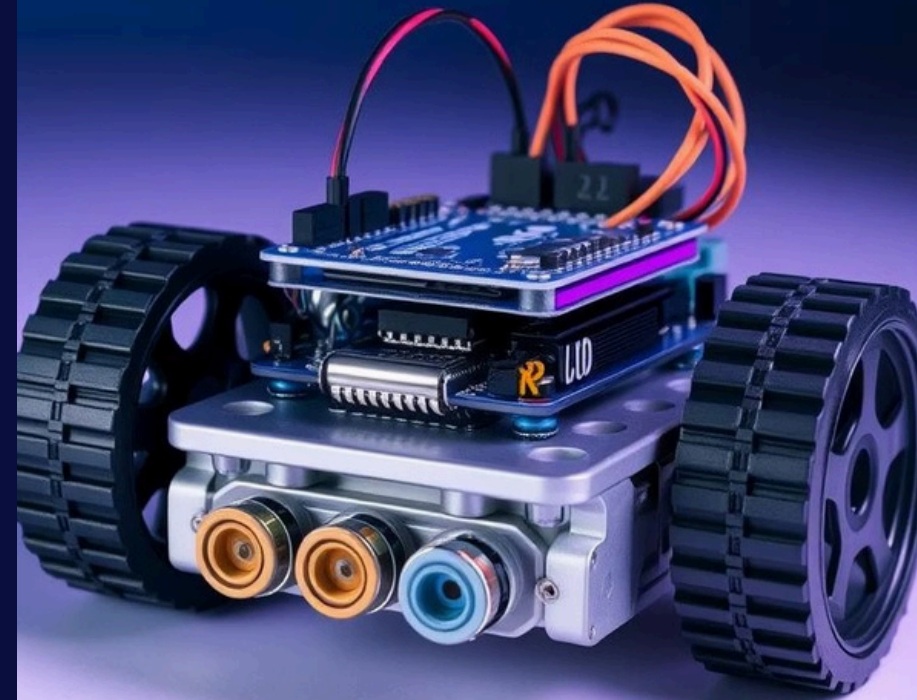
Détecte la distance aux obstacles avec précision.

Moteurs à courant continu

Permettent la propulsion et le contrôle de direction.

Pilote de moteur L298N

Commande la vitesse et la direction des moteurs.



Fonctionnement du Capteur à Ultrasons

- Émet une onde ultrasonore à 40 kHz
- Mesure le temps de retour de l'écho
- Calcule la distance via la vitesse du son (343 m/s)
- Portée de 2 cm à 400 cm
- Angle de détection restreint à 15 degrés

Contrôle des Moteurs à Courant Continu

Utilisation du pilote L298N

Commande la rotation et la vitesse des moteurs.

Direction contrôlée

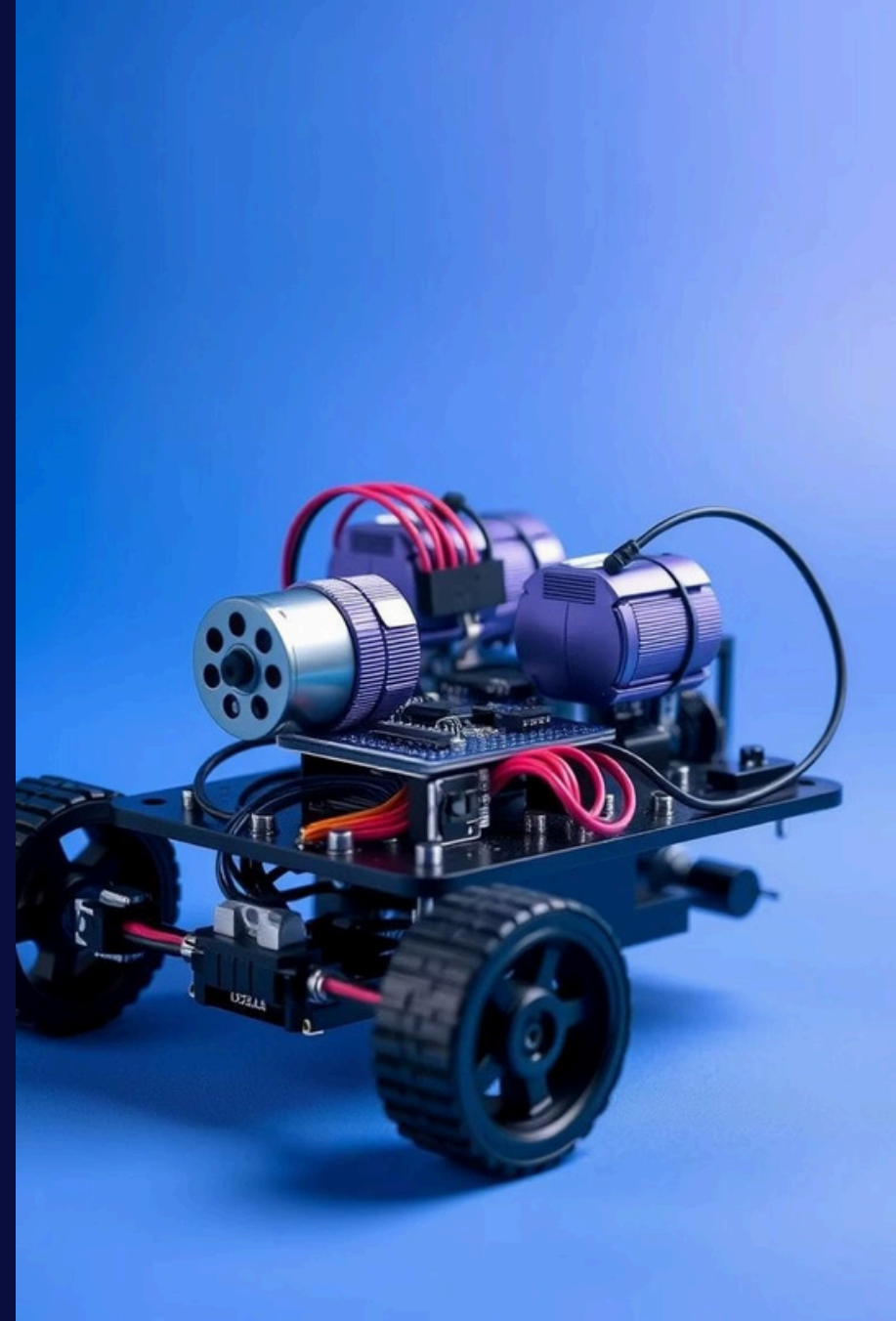
Avant, arrière, rotation gauche et droite facilitées.

Gestion de la vitesse

Modulation PWM pour une vitesse variable.

Alimentation séparée

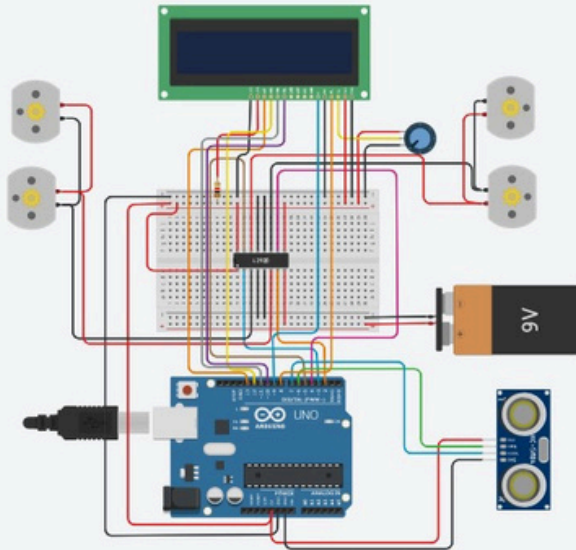
6V dédiée pour les moteurs, évitant les chutes de tension.



Intégration et Tests

- Assemblage soigné sur châssis motorisé
- Code téléversé via IDE Arduino
- Tests en conditions contrôlées
- Réglages du seuil et vitesses optimisés

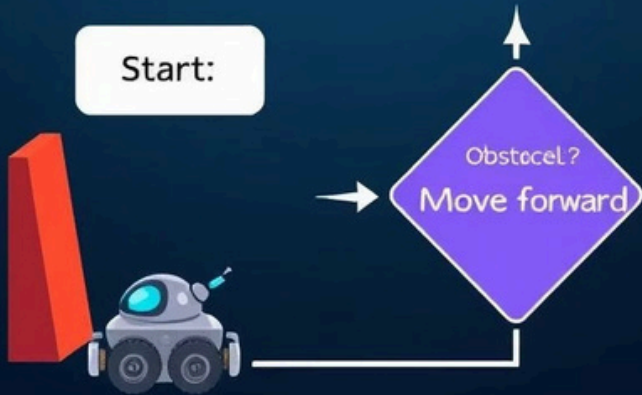
Voila notre carte



Composants utilisés :

- **Capteur ultrason HC-SR04 pour mesurer la distance :**
 - trigPin : broche de déclenchement (envoi du signal)
 - echoPin : broche de réception (réception de l'écho)
- **Pilote de moteur pour diriger le robot :**
 - lm1, lm2 : moteurs gauches
 - rm1, rm2 : moteurs droits
- Écran LCD 16x2 pour afficher les informations

Algorithme d'Évitement d'Obstacles



1

Lecture du capteur

Mesure continue de la distance aux obstacles.

2

Seuil critique

Distance seuil de 20 cm pour déclencher l'évitement.

3

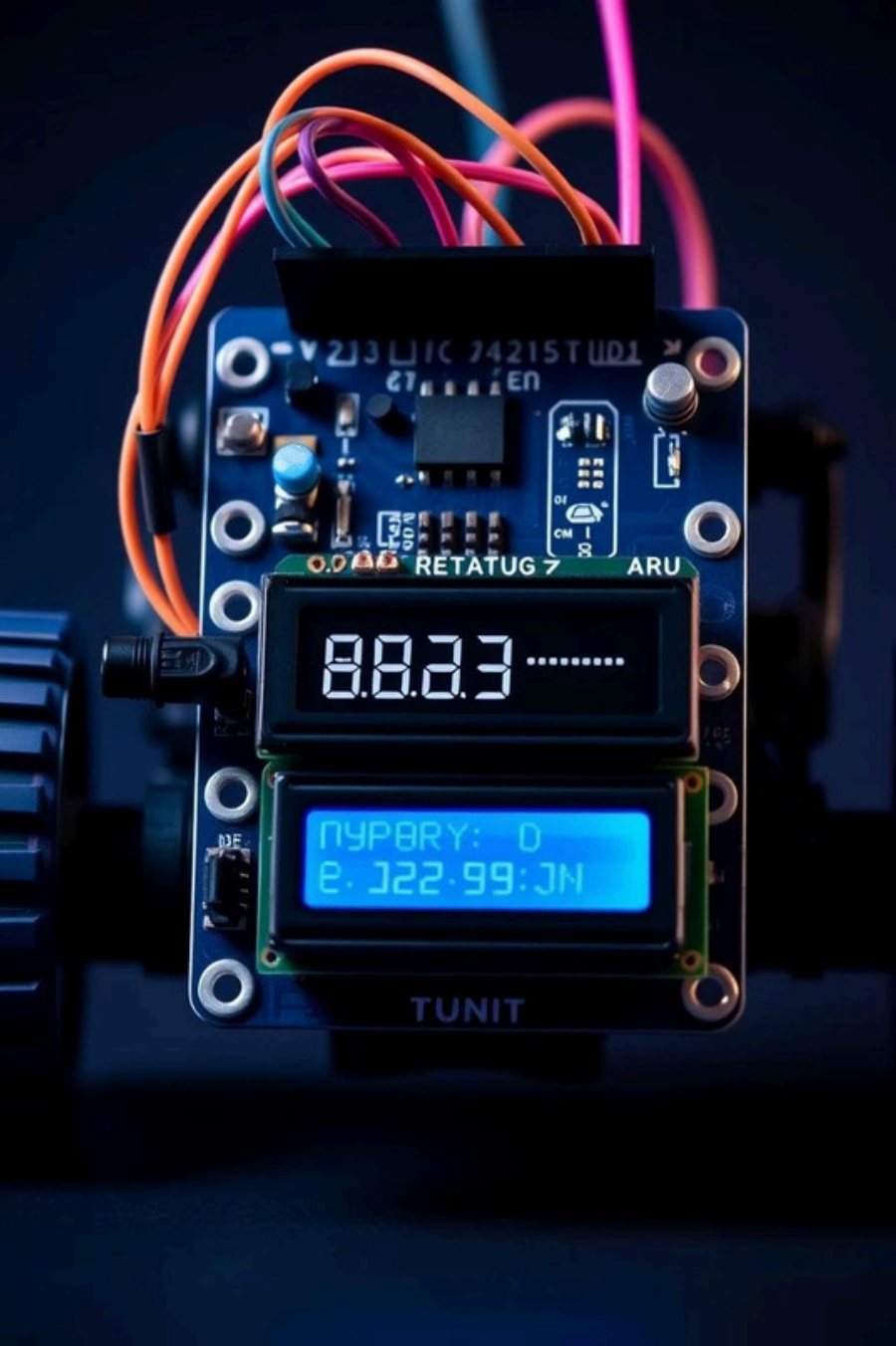
Action

Arrêt, rotation de 90° à gauche, vérification de l'espace.

4

Itération

Si obstacle encore présent, rotation à droite puis reprise.



Code Arduino

1 Définition des broches

Configuration du capteur et moteurs dans le code.

2 Initialisation

Paramètres prêts pour la boucle principale.

3 Boucle de contrôle

Lecture, décision, et pilotage des moteurs.

4 Fonctions

Utilisation de `digitalWrite()` et `analogWrite()` pour actions.

```

void stop_bot()
{
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("SOMETHING AHEAD");
  lcd.setCursor(0,1);
  lcd.print("STOP!!");

  digitalWrite(lm1,LOW);
  digitalWrite(lm2,LOW);
  digitalWrite(rm1,LOW);
  digitalWrite(rm2,LOW);
}

void stop_again()
{
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("BREAK FOR TURN");

  digitalWrite(lm1,LOW);
  digitalWrite(lm2,LOW);
  digitalWrite(rm1,LOW);
  digitalWrite(rm2,LOW);
}

void go_left()
{
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("TURNING LEFT");
  lcd.setCursor(0,1);
  lcd.print(cm);

  digitalWrite(lm1,LOW);
  digitalWrite(lm2,LOW);
  digitalWrite(rm1,HIGH);
}

```

```

// For Serial Monitor
Serial.print("Distance:CM ");
Serial.println(cm);
}

// Here are the functions that are used in the program

void go_straight()
{
  lcd.setCursor(0,0);
  lcd.print("NOTHING AHEAD");
  lcd.setCursor(0,1);
  lcd.print("MOVING FORWARD");

  digitalWrite(lm1,HIGH);
  digitalWrite(lm2,LOW);
  digitalWrite(rm1,HIGH);
  digitalWrite(rm2,LOW);
}

void go_back()
{
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("TAKING REVERSE");
  lcd.setCursor(0,1);
  lcd.print(cm);

  digitalWrite(lm2,HIGH);
  digitalWrite(lm1,LOW);
  digitalWrite(rm2,HIGH);
  digitalWrite(rm1,LOW);
}

void stop_bot()
{
  lcd.clear();
}

```



```

void loop()
{
    // the distance ahead using an ultrasonic sensor
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(5);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);

    // converting the time into a distance in Centimetre

    cm = duration*0.034/2;

    if(cm < 20)
    {
        stop_bot();
        delay(2000);

        go_back();
        delay(2000);

        stop_again();
        delay(1000);

        go_left();
        delay(1000);
    }

    else
    {
        go_straight();
        delay(1000);
    }
}

```

```

//Coded By TechLogy Hub
// This code is for a OBSTACLE AVOIDING ROBOT

#include <LiquidCrystal.h>
LiquidCrystal lcd(8, 9, 10, 11, 12, 13);

long cm, duration;
const int echoPin = 7;
const int trigPin = 6;

const int lm1 = 2;
const int lm2 = 3;
const int rm1 = 4;
const int rm2 = 5;

// Above is the motor driver pin connection. lm1, lm2, rm1 and
//rm2 are the digital input pins from arduino to the motor driver.

void setup()
{
    pinMode(lm1, OUTPUT);
    pinMode(lm2, OUTPUT);
    pinMode(rm1, OUTPUT);
    pinMode(rm2, OUTPUT);

    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    Serial.begin(9600);
    lcd.begin(16, 2);
}

```



Conclusion et Perspectives

Prototype fonctionnel

Robot autonome évitant les obstacles avec succès.

Améliorations

Multi-capteurs pour détection plus précise et fiable.

Algorithmes avancés

Implémentation possible d'algorithmes A* pour navigation.

Nouvelles fonctions

Cartographie d'environnement et adaptation en temps réel.